SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET
PROGRAMMING COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems. (L4, L5)

Assessment Tool : Industry Problem Task

Weightage: 40%

QUESTIONS: **Total Marks: 30**

• A hospital management system requires a **SOAP-based web service** to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

Design a **SOAP-based web service** for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

• An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use **WSDL (Web Services Description Language)** to describe the service.

Design the **WSDL structure** for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

• A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SOAP messages and error handling techniques. Explain how the corrected system ensures reliable communication.

Code of Conduct:

I E b i n A n t o n y P (1 9 2 3 7 3 0 1 1) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
		Innovative, practical, and	Logical and	Basic solution	Unclear or

Solution Design	25%	feasible solution aligned with industry standards	feasible solution with minor gaps	with limited feasibility	impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

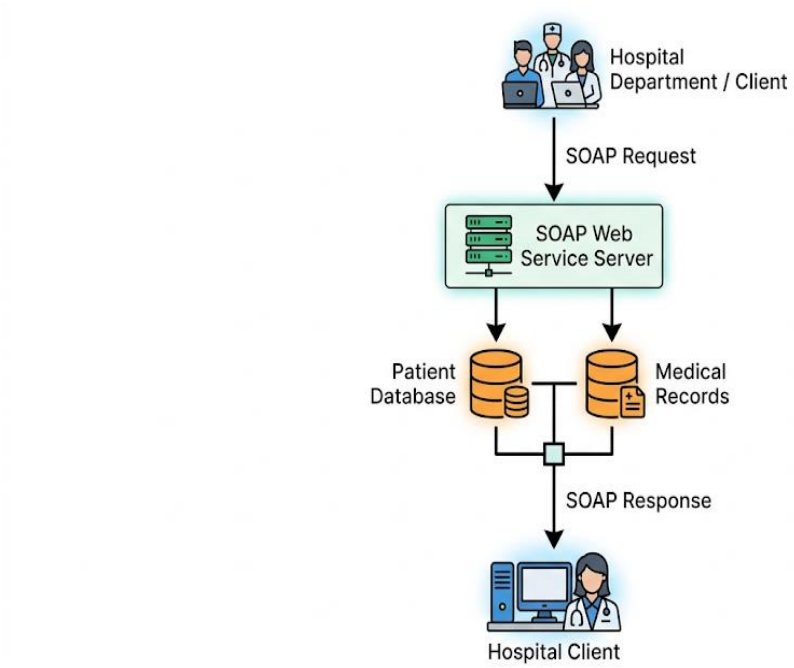
Industry Problem Task – Web Services

1. SOAP-Based Web Service for Hospital Management System

Introduction

A hospital management system can use a **SOAP (Simple Object Access Protocol) based web service** to enable secure and interoperable communication between departments such as reception, doctors, laboratory, pharmacy, billing, and medical records. SOAP uses XML messages, making communication platform-independent and suitable for distributed systems.

Proposed Architecture



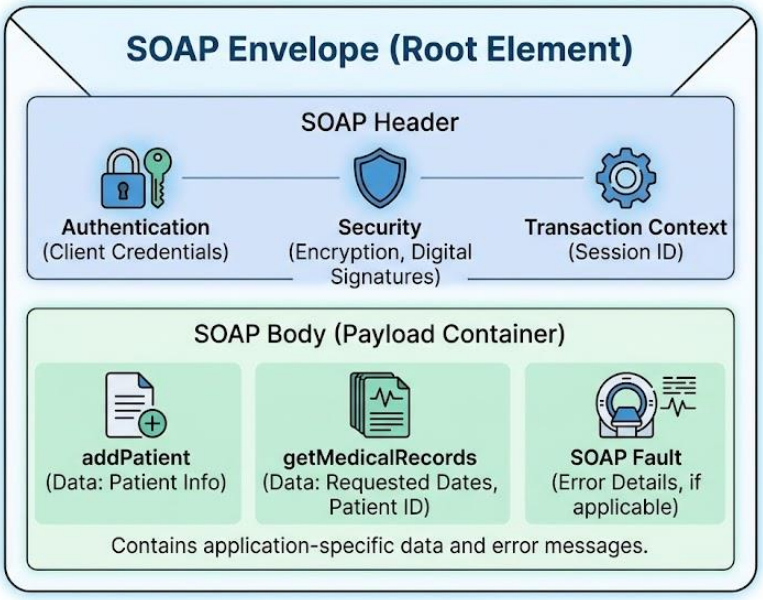
Main Operations

The service can provide the following operations:

Operation	Purpose	Input	Output
`addPatient()`	Register a new patient	Patient details	Patient ID/status
`getPatient()`	Retrieve patient details	Patient ID	Patient information
`scheduleAppointment()`	Schedule an appointment	Patient ID, doctor ID, date	Appointment status
`getAppointments()`	Retrieve appointments	Patient ID	Appointment list
`getMedicalRecords()`	Retrieve medical records	Patient ID	Medical history
`updateMedicalRecord()`	Update medical information	Patient ID, record details	Update status

SOAP Message Structure

A SOAP message consists mainly of an **Envelope, Header, and Body**.



1. SOAP Envelope

The Envelope is the root element of every SOAP message. It identifies the message as a SOAP message and contains the Header and Body.

2. SOAP Header

The Header contains optional information required for processing the request. In a hospital system, it can contain:

- Authentication information
- Digital signatures
- Security tokens
- Transaction information
- Routing information

3. SOAP Body

The Body contains the actual request or response. For example, an `addPatient` request can contain the patient's name, age, gender, contact information, and address.

Example SOAP Request

```
<soap:Envelope
 xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:hos="http://hospital.example.com/">

    <soap:Header>
        <hos:Authentication>
            <hos:UserID>doctor01</hos:UserID>
            <hos:Token>SECURE_TOKEN</hos:Token>
        </hos:Authentication>
    </soap:Header>

    <soap:Body>
        <hos:addPatient>
            <hos:PatientName>John</hos:PatientName>
            <hos:Age>35</hos:Age>
            <hos:Gender>Male</hos:Gender>
            <hos:Contact>9876543210</hos:Contact>
```

```
        </hos:addPatient>
    </soap:Body>

</soap:Envelope>
```

Example SOAP Response

```
<soap:Envelope
 xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

    <soap:Body>
        <addPatientResponse
          xmlns="http://hospital.example.com/">
            <PatientID>P1001</PatientID>
            <Status>Patient Added Successfully</Status>
        </addPatientResponse>
    </soap:Body>

</soap:Envelope>
```

Data Exchange Process

1. The hospital client creates a SOAP request.
2. Patient information is converted into XML.
3. The SOAP Envelope and Header are added.
4. The request is sent to the hospital web-service endpoint using HTTP/HTTPS.
5. The SOAP server receives and validates the request.
6. The server processes the requested operation.
7. The hospital database is accessed when required.
8. The server creates a SOAP response.
9. The response is returned to the client.
10. The client extracts the required information from the XML response.

Reliability and Security Justification

SOAP is suitable for the hospital environment because:

- **Platform independent:** Java, .NET, Python, and other platforms can communicate using SOAP.
- **XML-based:** Data is represented in a structured and standardized format.
- **Reliable communication:** SOAP supports standardized fault handling and can work with reliable messaging technologies.
- **Security:** HTTPS can protect data during transmission, while WS-Security can provide authentication, integrity, and message-level security.
- **Interoperability:** Different hospital departments and external systems can communicate even when developed using different technologies.
- **Structured error handling:** SOAP Fault messages provide standardized error information.

Therefore, SOAP provides a suitable mechanism for secure and interoperable exchange of sensitive hospital information.

2. WSDL Structure for an Enterprise Web Service

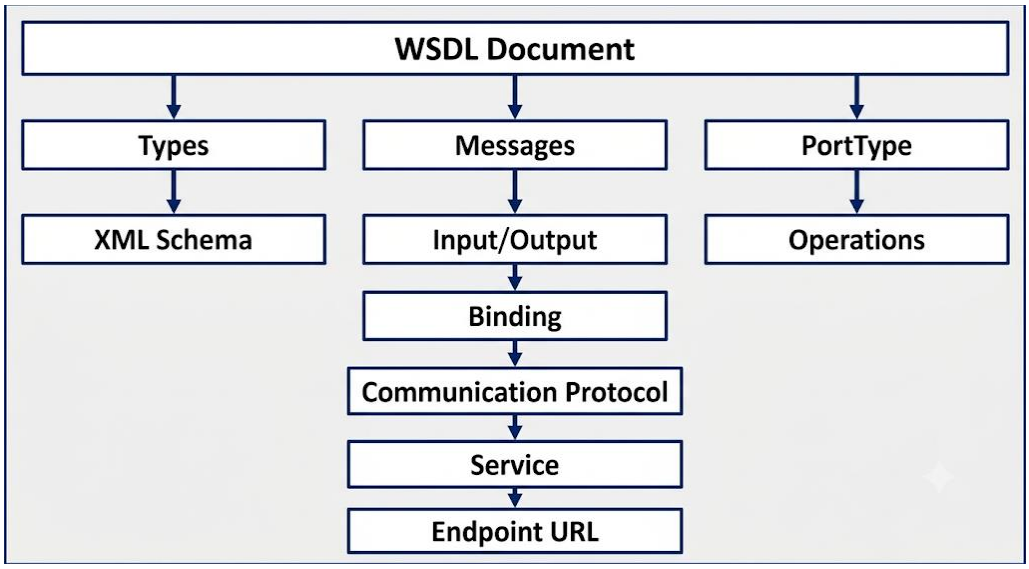
Introduction

WSDL (Web Services Description Language) is an XML-based language used to describe a web service. It tells client applications **what operations are available, what data is required, how messages are formatted, how communication takes place, and where the service is located**.

A WSDL document mainly contains:

1. Types
2. Message
3. PortType
4. Binding
5. Service

WSDL Architecture



1. Types

The `types` section defines the data types used by the service. XML Schema (XSD) can be used to define structured data.

Example:
```
<types>
    <xsd:schema
       xmlns:xsd="http://www.w3.org/2001/XMLSchema">

        <xsd:element name="getDataRequest">
            <xsd:complexType>
                <xsd:sequence>
                    <xsd:element name="id" type="xsd:int"/>
                </xsd:sequence>
            </xsd:complexType>
        </xsd:element>

    </xsd:schema>
</types>
```

2. Message

The `message` element defines the data exchanged between client and server.

```
<message name="GetDataRequest">
    <part name="parameters"
          element="tns:getDataRequest"/>
</message>

<message name="GetDataResponse">
    <part name="result"
          type="xsd:string"/>
</message>
```

Here, `GetDataRequest` represents the input and `GetDataResponse` represents the output.

3. PortType

The `portType` defines the abstract interface of the web service. It specifies the operations supported by the service.

```
<portType name="EnterprisePortType">

    <operation name="getData">
        <input message="tns:GetDataRequest"/>
        <output message="tns:GetDataResponse"/>
    </operation>

    <operation name="updateData">
        <input message="tns:UpdateDataRequest"/>
        <output message="tns:UpdateDataResponse"/>
    </operation>

</portType>
```

Thus, the client knows that the service provides `getData` and `updateData` operations.

4. Binding

The `binding` element specifies how the abstract operations are implemented. It can specify SOAP as the communication protocol.

```
<binding name="EnterpriseSOAPBinding"
         type="tns:EnterprisePortType">

    <soap:binding
       style="document"
       transport="http://schemas.xmlsoap.org/soap/http"/>

    <operation name="getData">
        <soap:operation soapAction="getData"/>
    </operation>

    <operation name="updateData">
        <soap:operation soapAction="updateData"/>
    </operation>

</binding>
```

5. Service

The `service` element specifies the actual service endpoint where the web service can be accessed.

```
<service name="EnterpriseService">

    <port name="EnterpriseSOAPPort"
          binding="tns:EnterpriseSOAPBinding">

        <soap:address
          location="https://example.com/EnterpriseService"/>

    </port>
</service>
```

Complete Simplified WSDL Structure

```
<definitions
   xmlns="http://schemas.xmlsoap.org/wsdl/"
    xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema"
    xmlns:tns="http://example.com/enterprise"
    targetNamespace="http://example.com/enterprise">

    <types>
        <!-- XML Schema definitions -->
    </types>

    <message name="GetDataRequest">
        <!-- Request parameters -->
    </message>

    <message name="GetDataResponse">
        <!-- Response parameters -->
    </message>

    <portType name="EnterprisePortType">
        <operation name="getData">
            <input message="tns:GetDataRequest"/>
            <output message="tns:GetDataResponse"/>
        </operation>
    </portType>

    <binding name="EnterpriseSOAPBinding"
             type="tns:EnterprisePortType">
        <!-- SOAP communication details -->
    </binding>

    <service name="EnterpriseService">
        <port name="EnterprisePort"
              binding="tns:EnterpriseSOAPBinding">
            <soap:address
              location="https://example.com/service"/>
        </port>
```

```
    </service>

</definitions>
```

Importance of WSDL in Service Integration

WSDL is important because it acts as a **contract between service providers and consumers**.

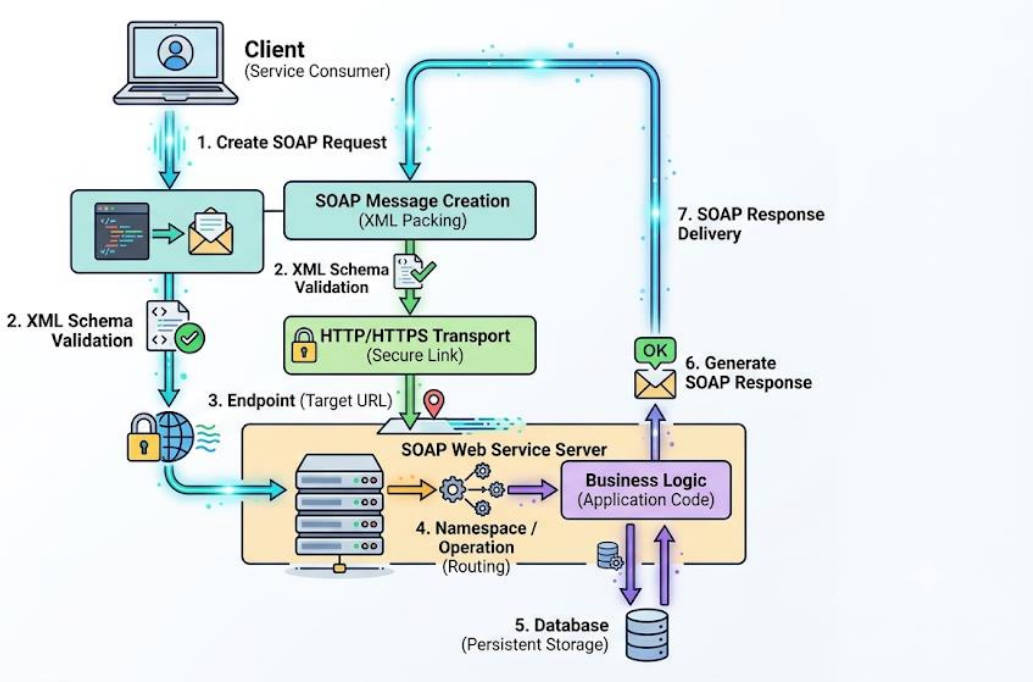
- It clearly defines available operations.
- It specifies input and output messages.
- It defines data types using XML Schema.
- It specifies communication protocols.
- It provides the service endpoint.
- It enables automatic client-code generation.
- It supports interoperability between different platforms.
- It reduces integration errors because both systems follow the same service contract.

Thus, WSDL allows external applications to understand and consume a web service without knowing its internal implementation.

3. Analysis and Debugging of SOAP Communication Problems

Problem Analysis

When SOAP requests are not processed correctly or responses are delayed or incorrect, the problem can occur at different stages of communication.



Possible Problems

1. SOAP Message Formatting Errors

Incorrect XML syntax can cause the server to reject the request.

Examples:

- Missing closing tags
- Incorrect XML nesting
- Invalid XML characters
- Incorrect SOAP Envelope
- Missing required elements

Solution: Validate the SOAP XML against the required schema and SOAP specification.

2. Incorrect Namespace

Namespaces identify XML elements and prevent naming conflicts. If the namespace used by the client differs from the namespace expected by the server, the operation may not be recognized.

Example:

```
<hos:getPatient
 xmlns:hos="http://hospital.example.com/">
```

If the server expects another namespace, the operation may fail.

Solution: Compare the namespaces in the SOAP request with those defined in the WSDL.

3. Incorrect Endpoint

The client may send requests to an incorrect URL.

Example:

`Expected:`
https://example.com/HospitalService

`Incorrect:`
https://example.com/HospitalServices

Solution: Verify the endpoint URL specified in the WSDL and ensure that the server is running and reachable.

4. Incorrect SOAPAction

The SOAPAction may not match the operation expected by the server.

```
SOAPAction: getPatient
```

If the server expects `getMedicalRecords`, the request may not be processed correctly.

Solution: Check the SOAPAction value against the WSDL binding.

5. Data Type Mismatch

The client may send a string when the server expects an integer.

Example:

```
<PatientID>ABC123</PatientID>
```

when the schema expects:

```
<xsd:element name="PatientID"
             type="xsd:int"/>
```

Solution: Validate the request against the XML Schema defined in the WSDL.

6. Authentication and Security Problems

A request can fail if authentication tokens, certificates, or security headers are missing or invalid.

Solution:

- Check authentication credentials.
- Validate security headers.
- Verify certificates.
- Use HTTPS.
- Check WS-Security configuration.

7. Network and Timeout Problems

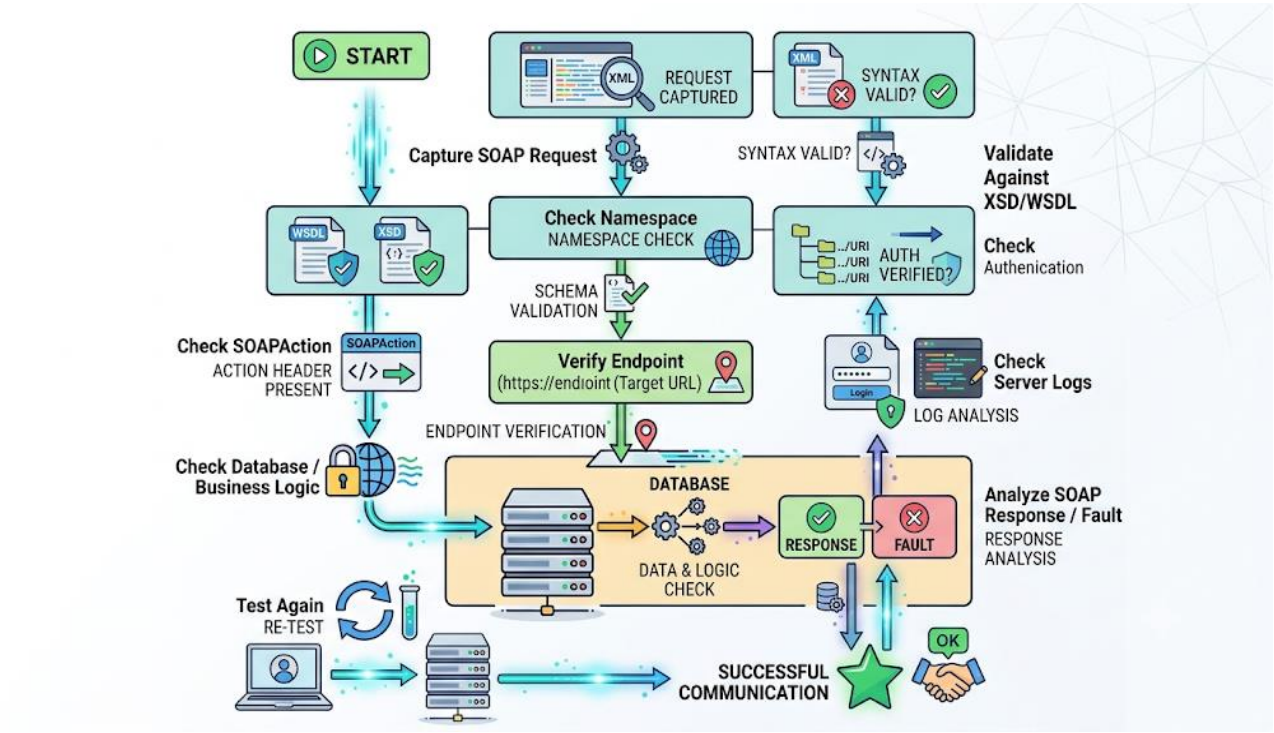
Delayed responses may be caused by:

- Network connectivity problems
- Server overload
- Incorrect timeout settings
- Database delays
- Firewall or proxy problems

Solution: Check network connectivity, server logs, response time, firewall rules, and database performance.

Debugging Approach

A systematic debugging procedure can be followed:



SOAP Fault Handling

SOAP provides a standard mechanism called **SOAP Fault** for reporting errors.

Example:

```
<soap:Envelope
 xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

    <soap:Body>

        <soap:Fault>
```

```
            <faultcode>
                soap:Client
            </faultcode>

            <faultstring>
                Invalid Patient ID
            </faultstring>

            <detail>
                Patient ID must be a valid integer.
            </detail>

        </soap:Fault>

    </soap:Body>

</soap:Envelope>
```

The client can inspect the Fault and provide an appropriate error message instead of treating the response as valid data.

Tools and Techniques for Debugging

The following techniques can be used:

- Capture and inspect raw SOAP requests and responses.
- Validate XML against the WSDL/XSD.
- Compare namespaces with the WSDL.
- Verify the endpoint URL.
- Check SOAPAction and HTTP headers.
- Examine server and application logs.
- Test the service using SOAP testing tools such as SoapUI.
- Check HTTP status codes.
- Monitor network connectivity and timeout values.
- Verify authentication and security configuration.
- Test individual operations separately.

How the Corrected System Ensures Reliability

After correction, the system should:

1. Generate valid XML messages.
2. Follow the exact SOAP Envelope structure.
3. Use the correct namespaces.
4. Follow the WSDL-defined operations.
5. Validate data using XML Schema.
6. Send requests to the correct endpoint.
7. Use appropriate authentication and HTTPS.
8. Handle SOAP Faults properly.
9. Log errors for troubleshooting.
10. Use suitable timeout and retry mechanisms where appropriate.

Conclusion

SOAP, WSDL, and XML Schema work together to provide a structured and interoperable web-service architecture. **SOAP** provides standardized XML-based message exchange, **WSDL** defines the service contract, and **XML Schema** ensures that exchanged data follows a valid structure. Proper namespace management, endpoint configuration, message validation, security, logging, and SOAP Fault handling help achieve reliable communication between distributed systems.